

Version Control System

Nazih Errahel

erraheln@ex.istu.edu

OUTLINE

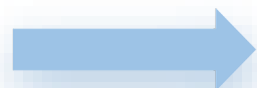
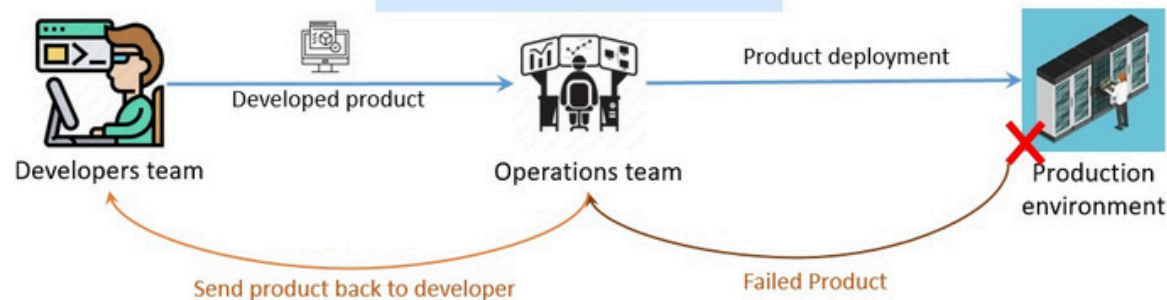
- Version Control System
- VCS Types & Goals
- VCS software tools
- Git History & Features
- Git --Installation & Configuration
- Git Repository and States
- Some more git concepts....(Pulling vs cloning Repository, Branching, Forking, Merging, Rebase)

A quick recap...

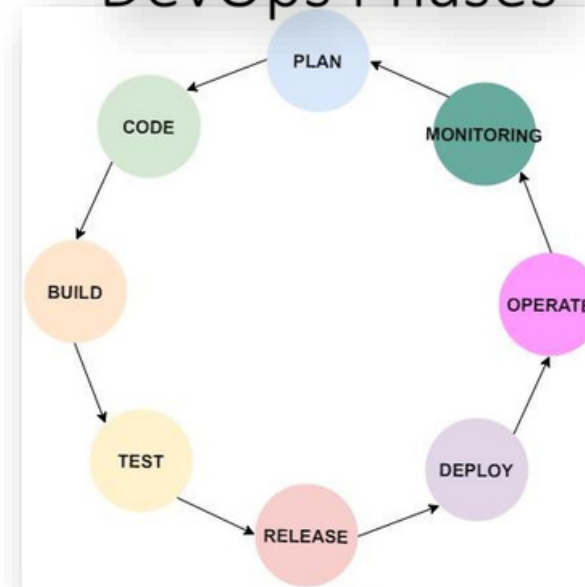
Software Development Life Cycle (SDLC)

Agile Model:
Disadvantages:

Why DevOps?



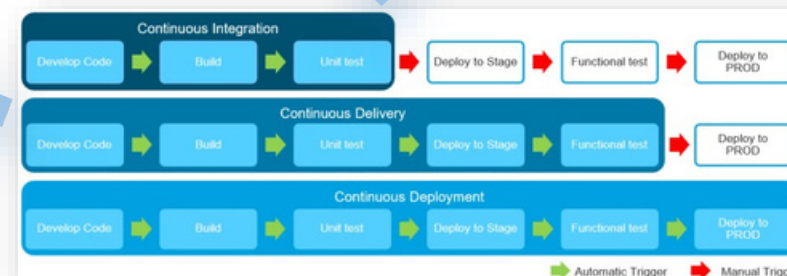
DevOps Phases



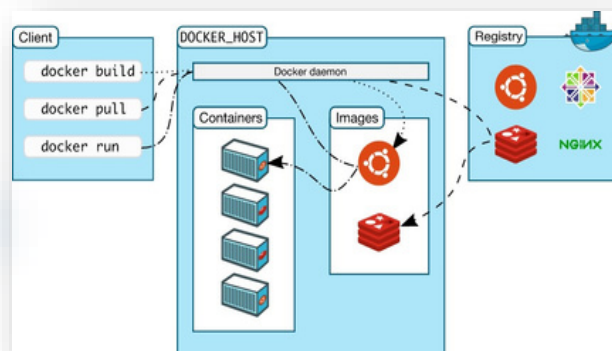
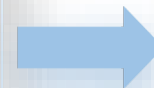
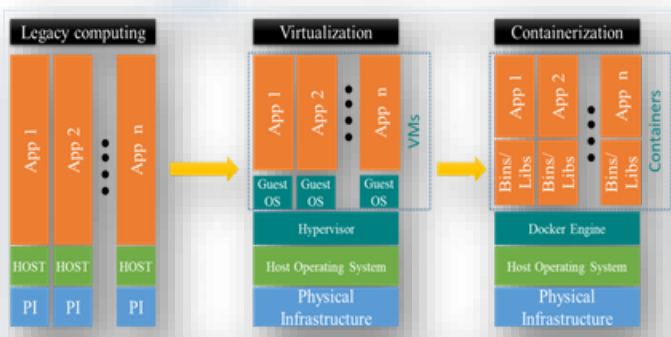
Why are we jumping to Cloud computing now?

4% of respondents currently don't use a cloud provider:
- DevSecOps Survey 2022 by GitLab

21% of respondents indicated not using the cloud, and instead used a data center or onpremises solution:
- State of DevOps 2021 by Google

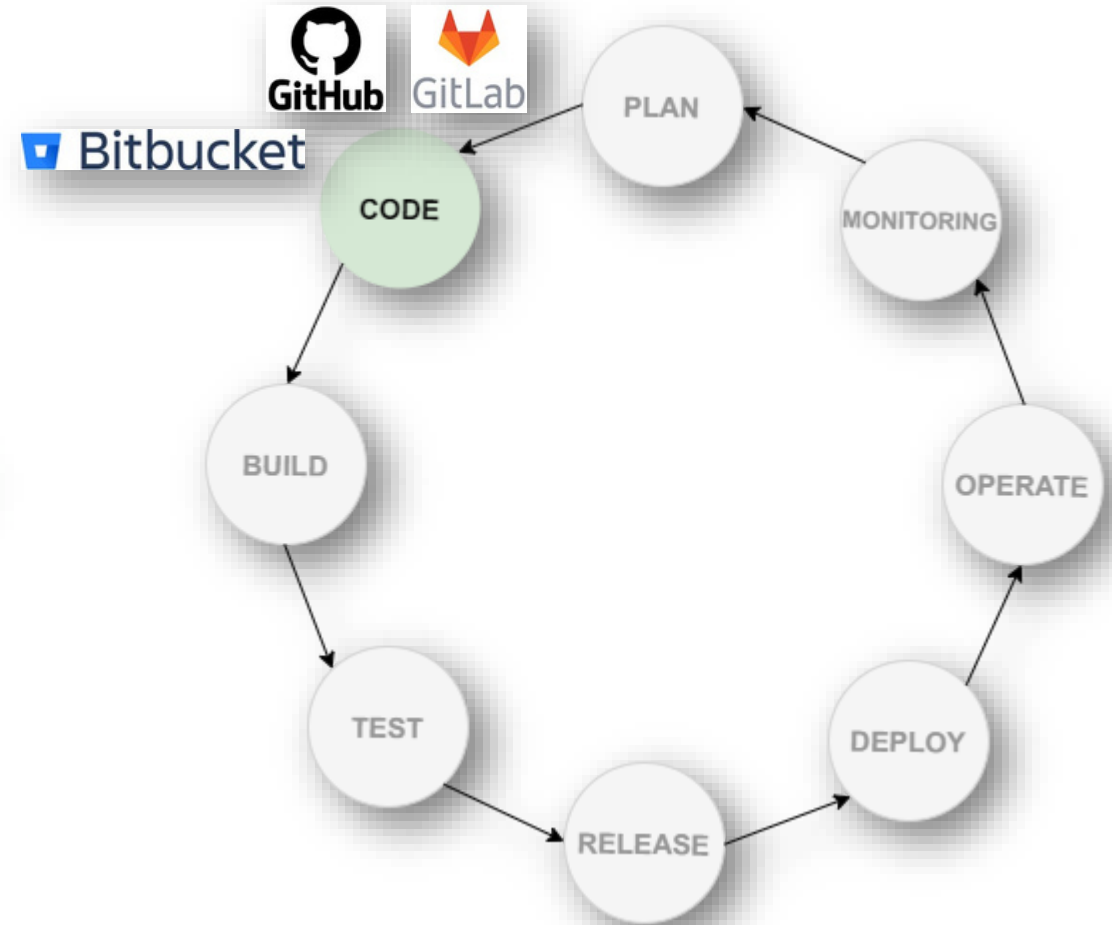
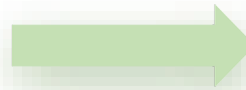
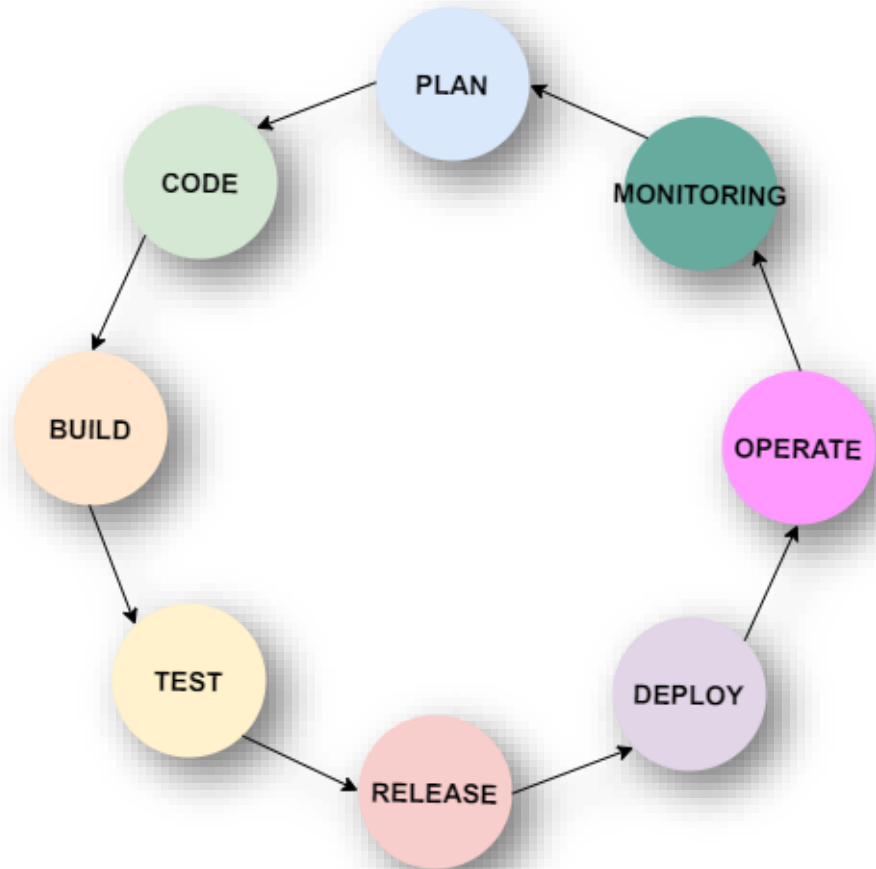


Img src: <https://medium.com/ibm-garage/devops-adoption-approach-plan-and-design-be3d1ba67c8>



Version Control System (VCS)

DevOps Phases



Version Control System

Version Control System -Introduction

help.html -V1

```
<h1>File: Help.html</h1>
<h1>Hello from Software development technologies Course.</h1>
<h2>Nazih Errahel</h2>
<h3>06 Mar 2026</h3>
```



help.html -V2

```
<h1>File: Help.html</h1>
<h1>Hello from Software development technologies Course.</h1>
<h2>Nazih Errahel</h2>
<h2>Institute of B-BRICKS</h2>
<h2>Irkutsk national research technical university </h2>
<h2>Russia</h2>

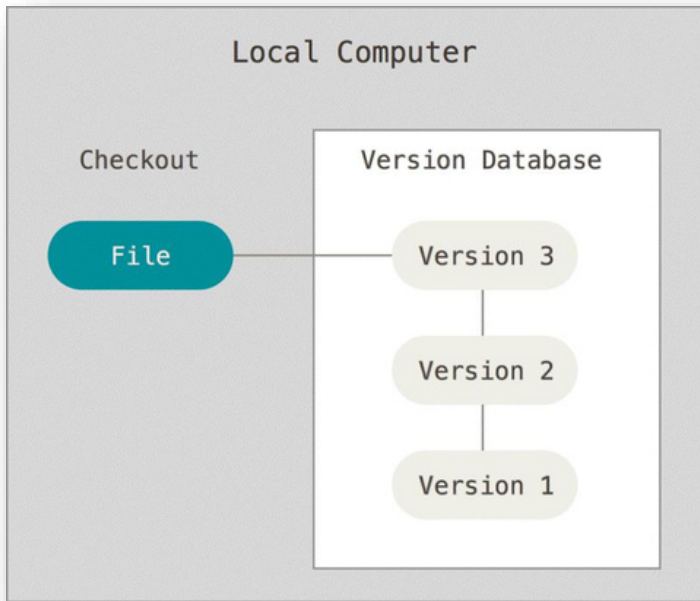
<h3>06 Mar 2026</h3>
```



help.html -V3

```
<h1>Hello from Software development technologies Course.</h1>
<h2>Nazih Errahel</h2>
<h2>Institute of B-BRICKS</h2>
<h2>Irkutsk national research technical university </h2>
<h2>Russia</h2>

<h2>Schedule:</h2>
<pre>
    Lectures:  Friday : 8:15-9:45      : Room no. 208
    Practice:  Friday : 10:00-11:30    : Room no. 208
</pre>
```



Src: <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

Version Control System -Introduction

“Version control, also known as source control, is the practice of tracking and managing changes to software code.”

*“Version control systems are software tools that help software teams **manage changes** to source code over time.”*



Create a file, **edit** the file, **save** the file, **modify** the file, **delete** the file.....and many more operations

- Bitbucket

Using a VCS also generally means that if you lose files, you can easily recover

But why to manage changes ?

*“Version control systems are software tools that help software teams **manage changes** to source code over time.”*

- Bitbucket

Answering *Who, Why, What, When, for Whom*

Some benefits of such VCS

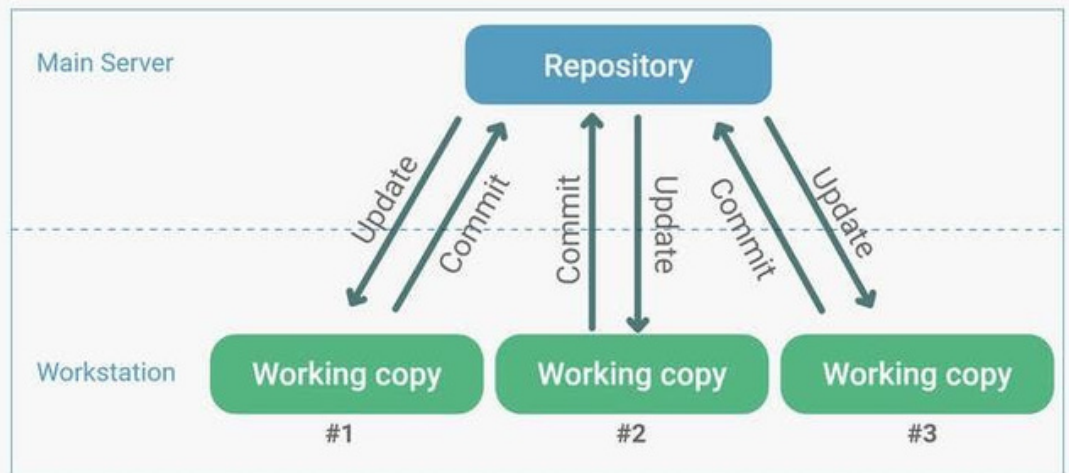
- Concurrency (e.g. Google Doc vs working on same docx file through email)
- Preserving collaborative history
- Rollback and undo changes
- Offsite source code backup
- Branching and merging
- Traceability
- Conflict resolution

VCS types

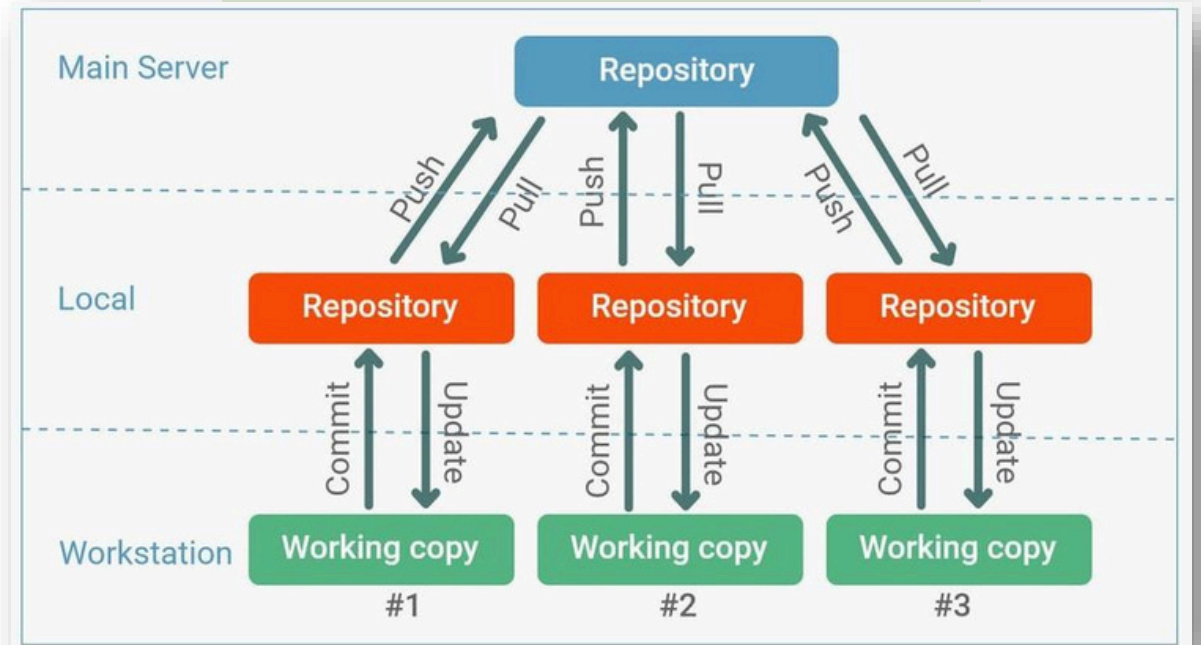
Local Version Control Systems



Centralized Version Control Systems



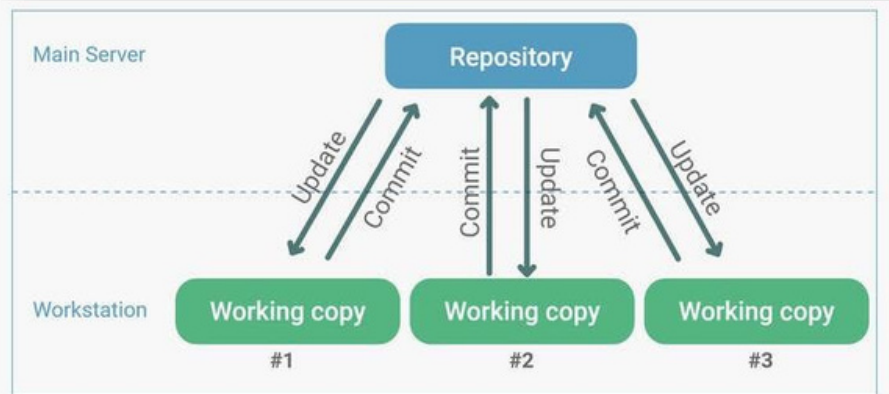
Distributed Version Control Systems



VCS types

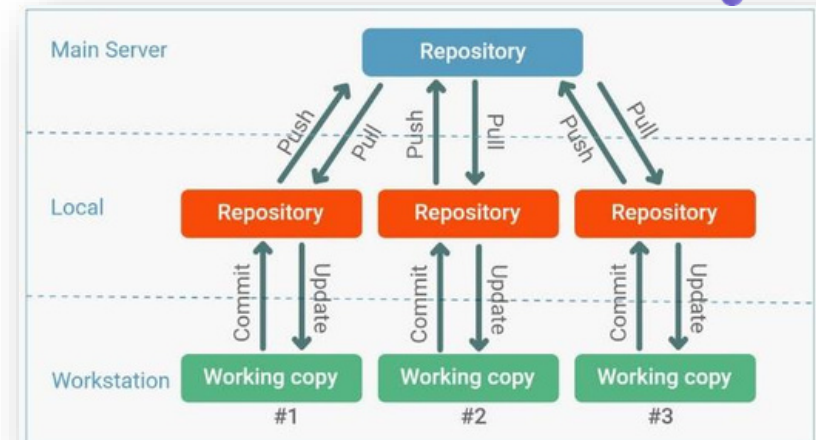
Centralized Version Control Systems

- **Location:** central server
- Need to be online, not locally available
- Need to connect to the central server for every command
- single point of failure
- Rely on the backups



Distributed Version Control Systems

- **Location:** Distributed in nature with local copy of entire history
- not necessary to be online
- Only required during pushing or pulling the code
- Better performance
- Local backup, if remote crashes
- Remote backup, if local crashes



Git

- A good implementation of above VCS idea

Git -Features

- **Distributed** Version Control System
- Git stores data as a series of snapshots
- Great branching model that adapts well to most workflows
- Fast and reliable
- Keeps a complete history
- Disaster recovery friendly
- Open Source

Git --Installation

- Windows OS:
 - <https://git-scm.com/download/win>
- macOS:
 - brew install git
 - ref: <https://git-scm.com/download/mac>
- Linux and Unix
 - Ubuntu:
 - apt-get install git
 - Centos:
 - yum install git
 - Ref: <https://git-scm.com/download/linux>
- git version with `git --version` command
- Enter `git` to see the list of available options/commands
- For any help enter `git help <command name>`
`git help init`

Git --Configuration

Project level

- only available for the current project
- stored in **.git/config** in the project's directory

Override

Global level

- Available for all projects for the current user
- Stored in **~/.gitconfig**

Override

System level

- Available for all the users/projects
- Stored in **/etc/gitconfig**

Git –Configuration commands

Project level

- only available for the current project
- stored in `.git/config` in the project's directory

Sample command:
`git config user.name "naziherrahel"`

Override

Global level

- Available for all projects for the current user
- Stored in `~/.gitconfig`

Sample command:
`git config --global user.name "naziherrahel"`

Level

Key

Value

Override

System level

- Available for all the users/projects
- Stored in `/etc/gitconfig`

Sample command:
`git config --system user.name "naziherrahel"`

Level

Key

Value

Git –Configuration commands

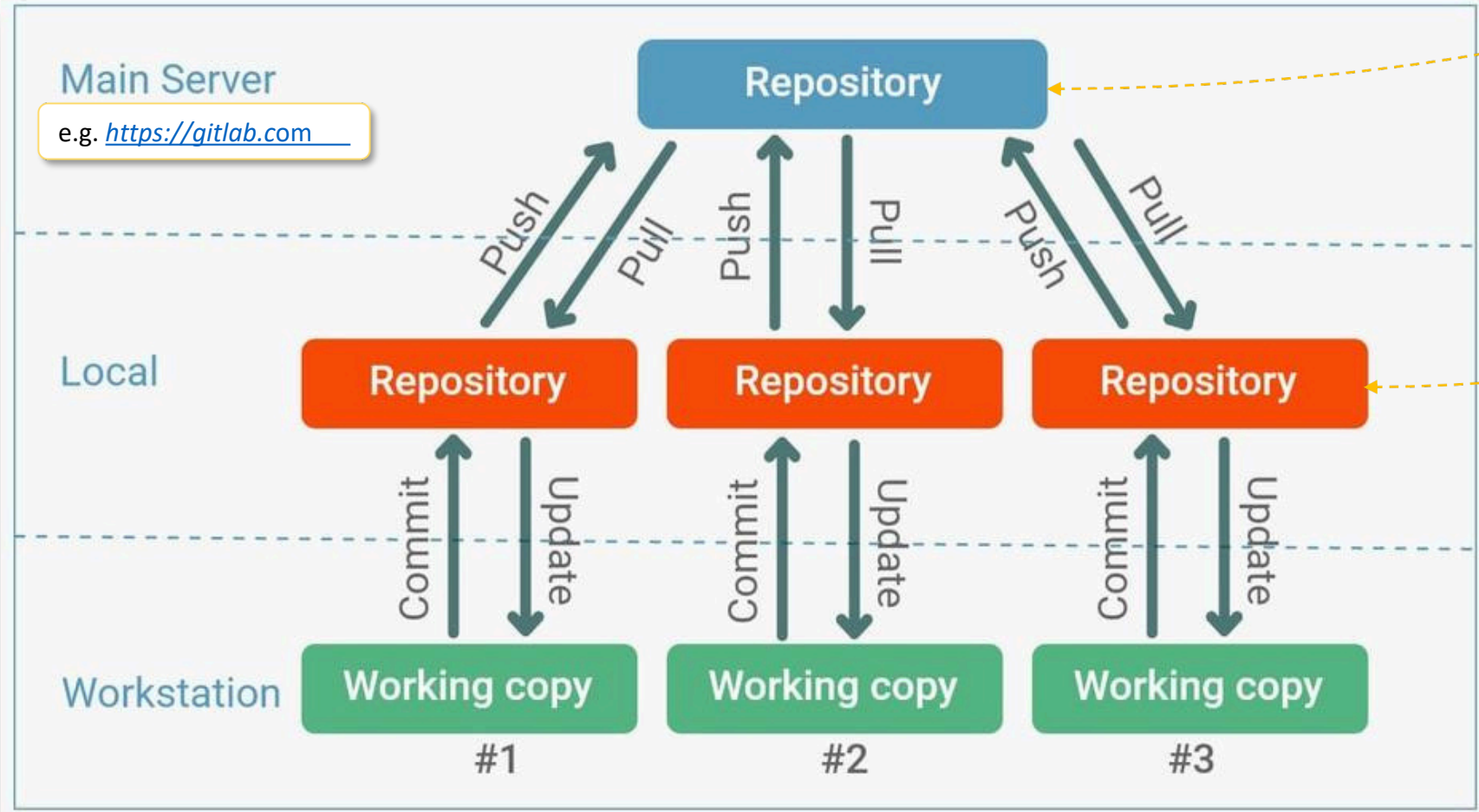
Some more configuration related commands

- list all
 - `git config -l`
- open an editor to edit the config file
 - `git config -e`
- Set the value of a specific key:
 - For local : `git config user.name "naziherrahel"`
 - For Global level: `git config--global user.name "naziherrahel"`
 - For system level: `git config--system user.name "naziherrahel"`
- get value: name [value-pattern]
 - `git config <key>`

Git Repository

Distributed Version Control Systems

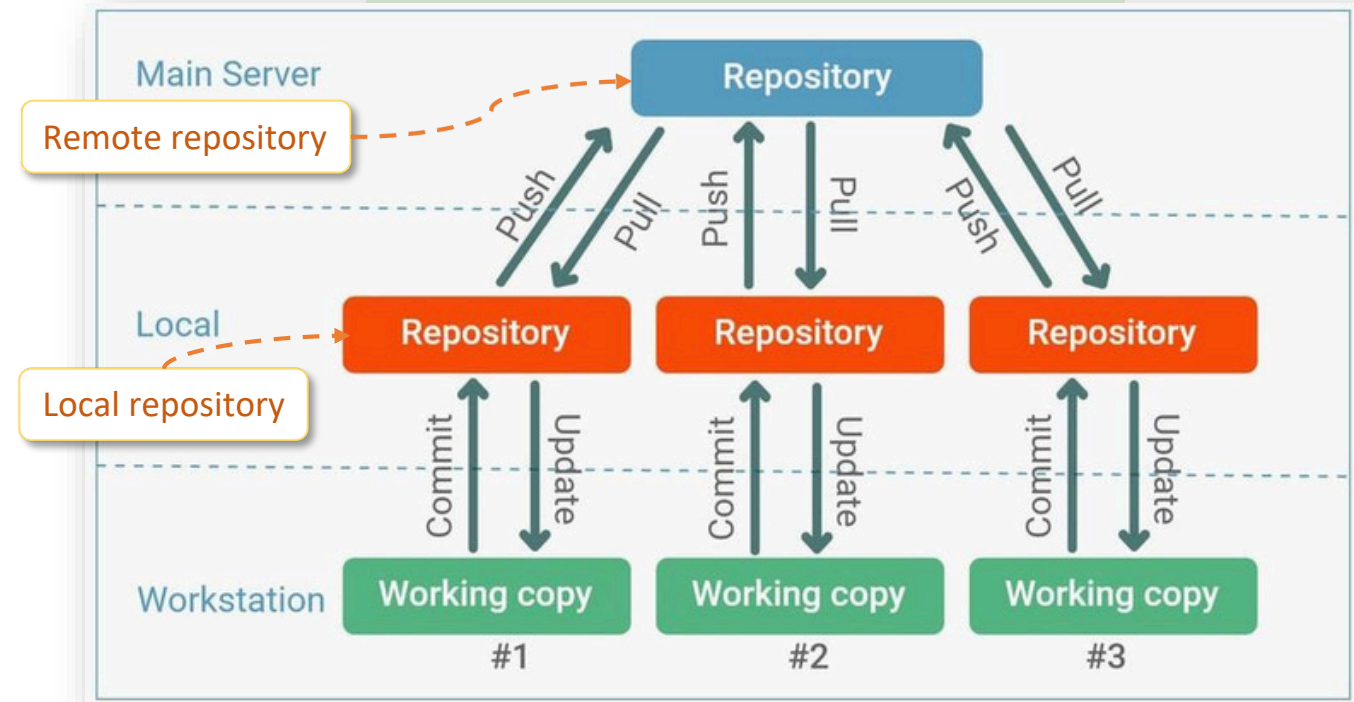
e.g. https://gitlab.com/naziherrahel/software_develo..



e.g. D:/gitlab/software_develo..

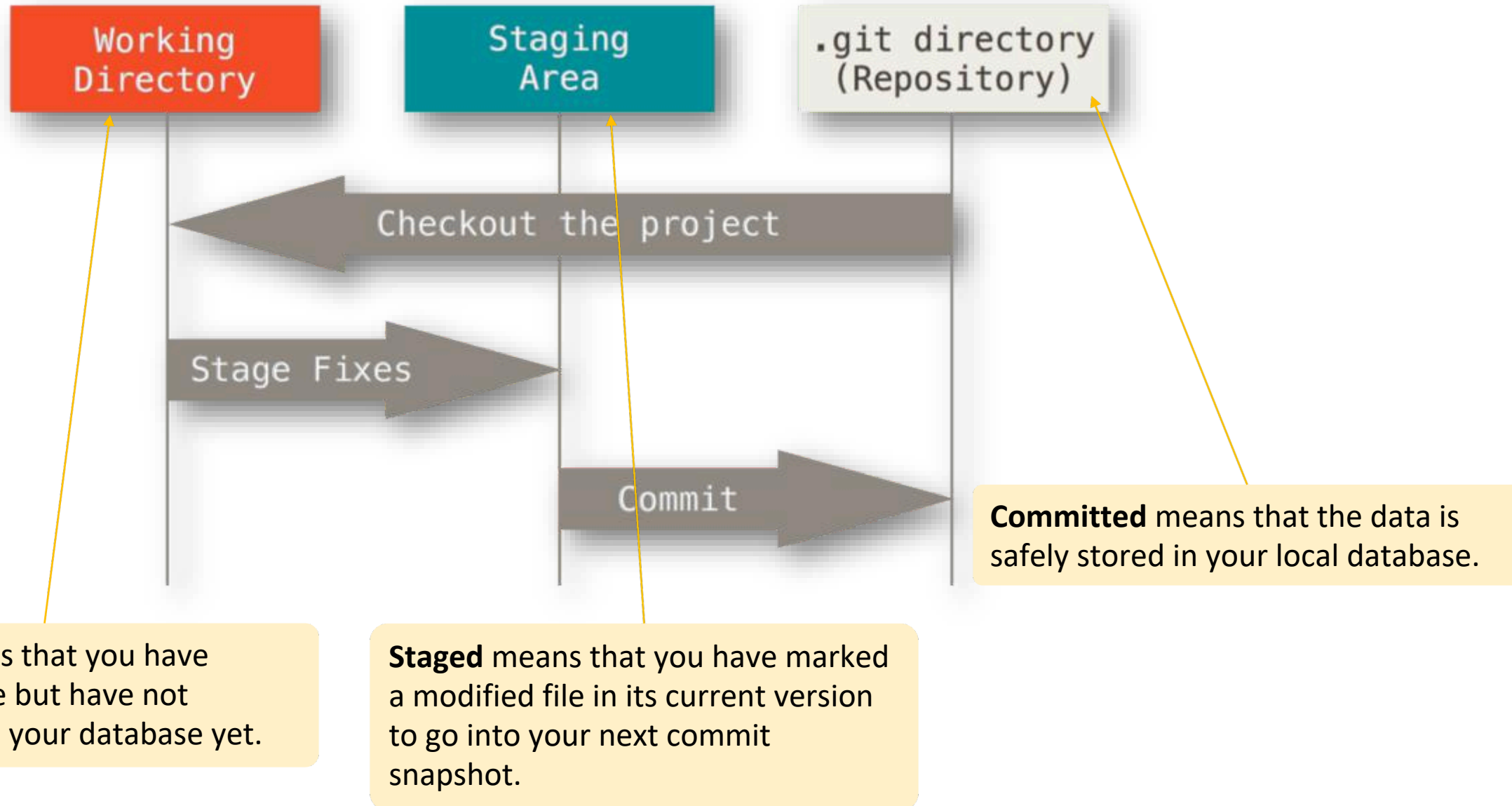
Git Repository

Distributed Version Control Systems

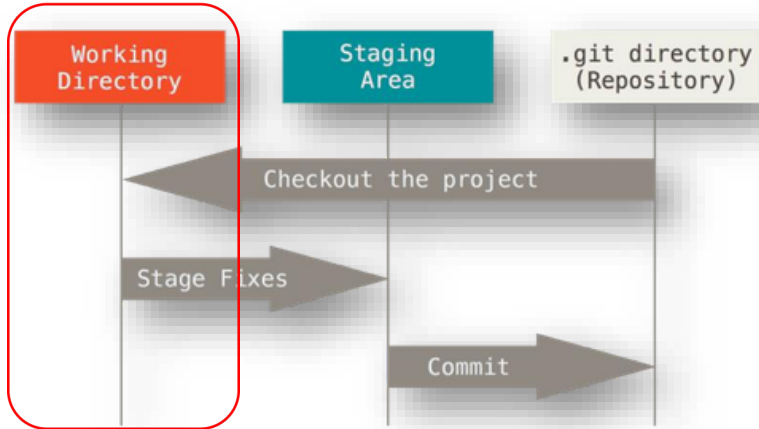


- Keep meta info
 - What you change
 - Change time
 - Who changed
 - Etc....
- **New** repo can be created
 - on Remote server Or Local
 - storage system
 - **Local:** use *git init* command
 - **Remote:** using GUI
- **Cloning an existing repository**
 - Using *git clone <repo url>*
 - *git clone*
https://gitlab.com/naziherrahel/Software_development_technologies.git

Git States



Git states -Staging



Lets have a quick walk through

Create a new repository:

```
12/10/2021 15:49.26 /mnt/d > mkdir testRepo
12/10/2021 15:49.34 /mnt/d > cd testRepo/
12/10/2021 15:49.36 /mnt/d/testRepo > git init
Initialized empty Git repository in /drives/d/testRepo/.git/
```

Check the status

```
12/10/2021 15:49.40 /mnt/d/testRepo > git status
On branch master
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

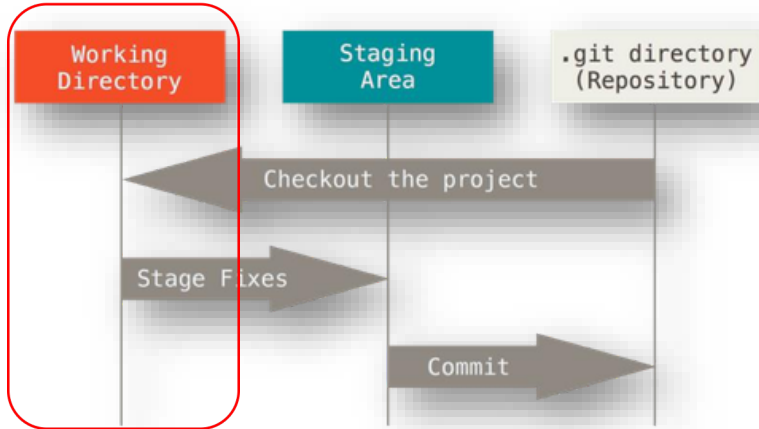
Create a new file with some random text

```
/mnt/d/testRepo > master > echo "this is the first change." > readme.md
```

Check the status of the repo

```
12/10/2021 15:54.08 /mnt/d/testRepo > master > git status
On branch master
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  readme.md
nothing added to commit but untracked files present (use "git add" to track)
```

Git states -Staging

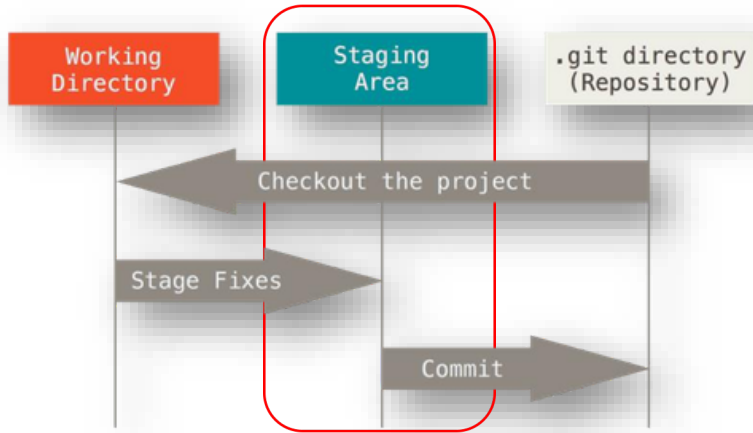


Lets have a quick walkt hrough

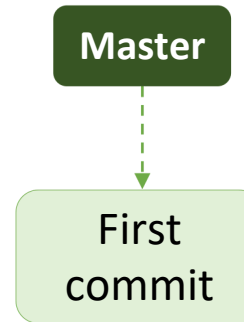
Adding the changes to staging area

```
12/10/2021 15:54.14 /mnt/d/testRepo master git add .
12/10/2021 15:57.17 /mnt/d/testRepo master git status
On branch master
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.md
```


Git states -Commit



Lets have a quick walk through



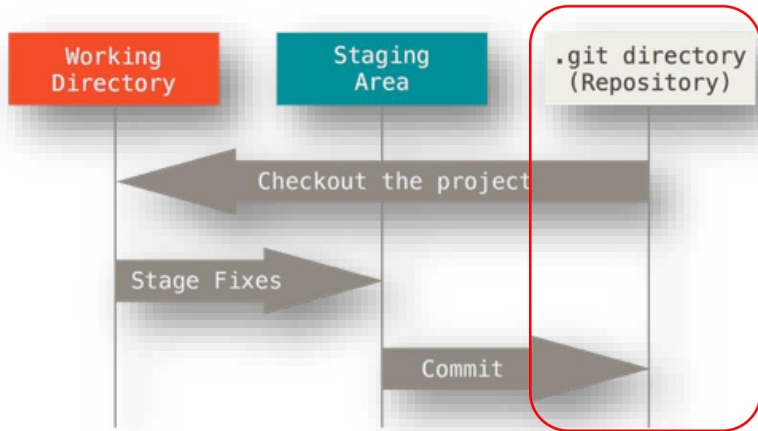
Committing the changes

```
12/10/2021 16:01.22 /mnt/d/testRepo master git commit -m "first change.."
[master (root-commit) 1e11daa] first change..
1 file changed, 1 insertion(+)
create mode 100644 readme.md
```

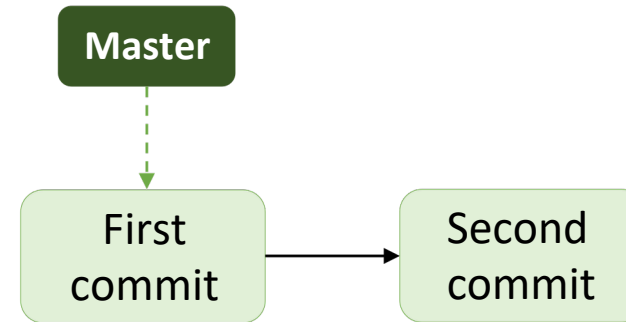
Lets again see the status

```
12/10/2021 16:05.10 /mnt/d/testRepo master git status
On branch master
nothing to commit, working tree clean
```

Git states –modify and Commit again



Lets have a quick walkt hrough

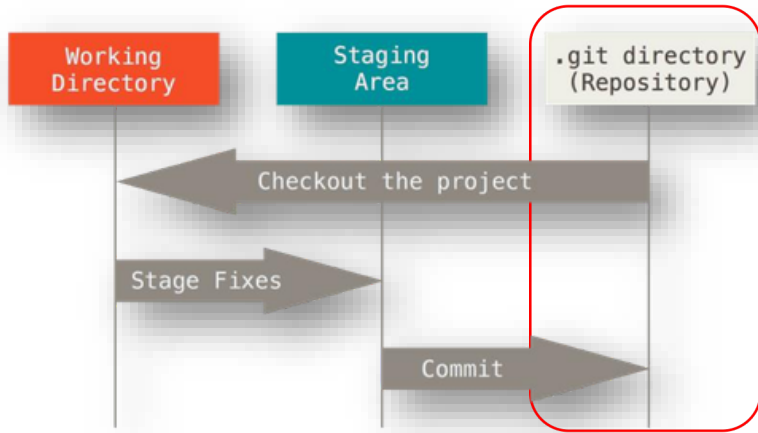


The second commit stores a pointer to the first commit.

Committing the changes

```
12/10/2021 22:49.52 /mnt/d/testRepo master git commit -m "second change.."
[master e0cf918] second change..
1 file changed, 1 insertion(+)
```


Git logs –Commit history



Committing the changes

```
12/10/2021 16:01:22 /mnt/d/testRepo master git commit -m "first change.."
[master (root-commit) 1e11daa] first change..
1 file changed, 1 insertion(+)
create mode 100644 readme.md
```

Will discuss in details in next lecture

Lets see the commits

```
commit db625752dfdc05f357c191e5a68e2dbb1f6f7295 (HEAD -> main)
Author: naziherrahel <naziherrahel@gmail.com>
Date: Fri Feb 27 10:10:35 2026 +0800

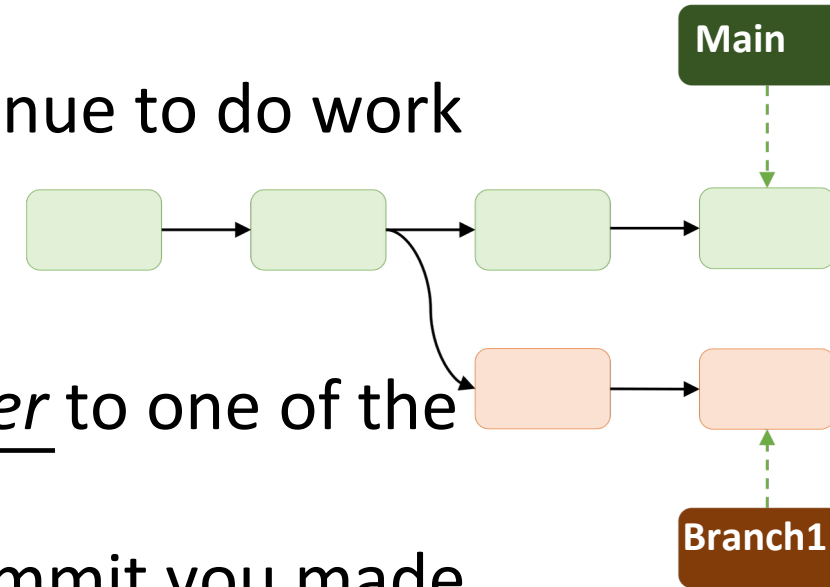
upload lab4
```

Branching

Branching

Branch

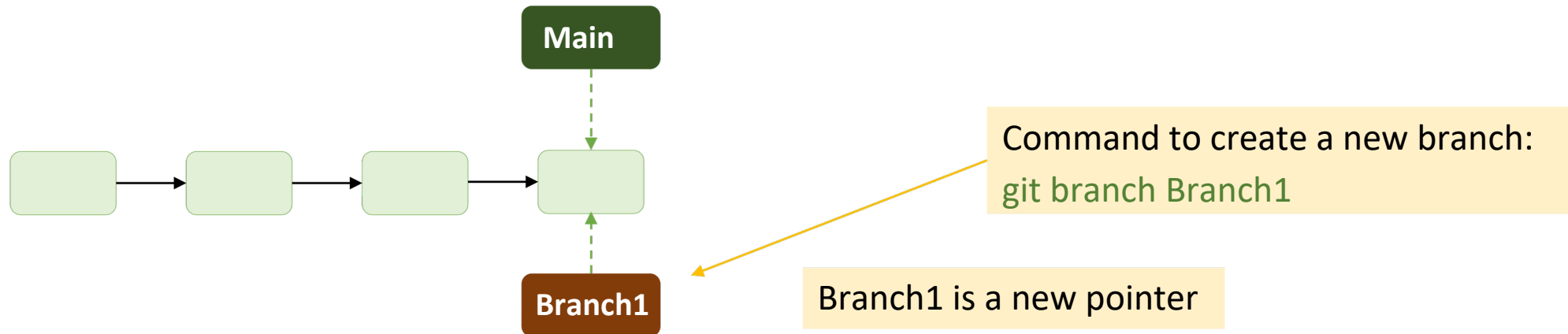
- A branch is a version of a project's working tree.
- diverge from the main line of development and continue to do work without messing with that main line.
- The default branch name in Git is **master**(or **main**)
- A branch in Git is simply a lightweight movable pointer to one of the commits.
- The **main/master** branch always points to the last commit you made.
- upon each new **commit**, the master branch pointer moves forward automatically.
- **master** branch in Git is not a special branch. It is exactly like any other branch.



Branching –create a new one

Branch

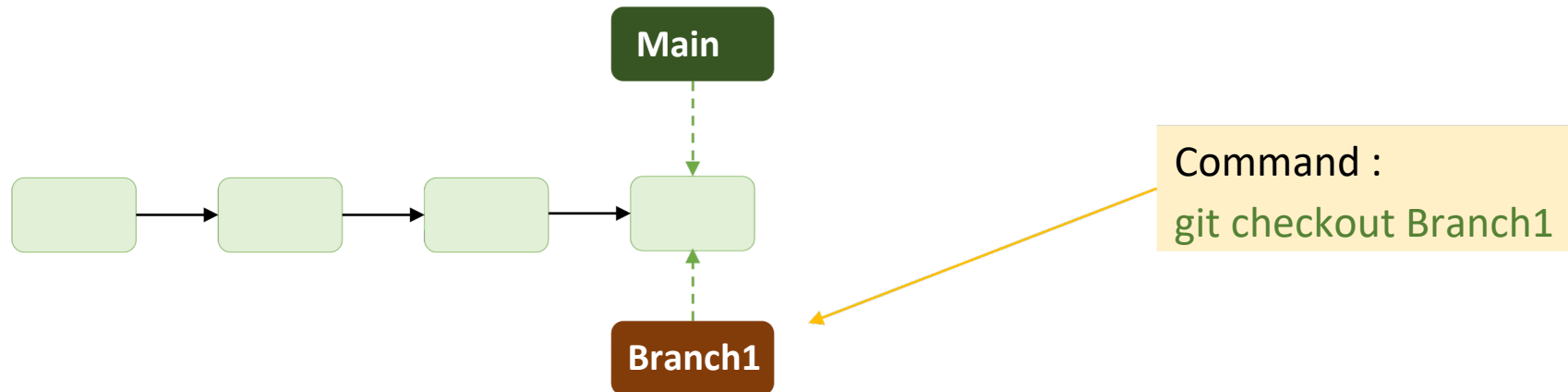
- A new branch means -> a new pointer



Branching –working on new branch

Branch

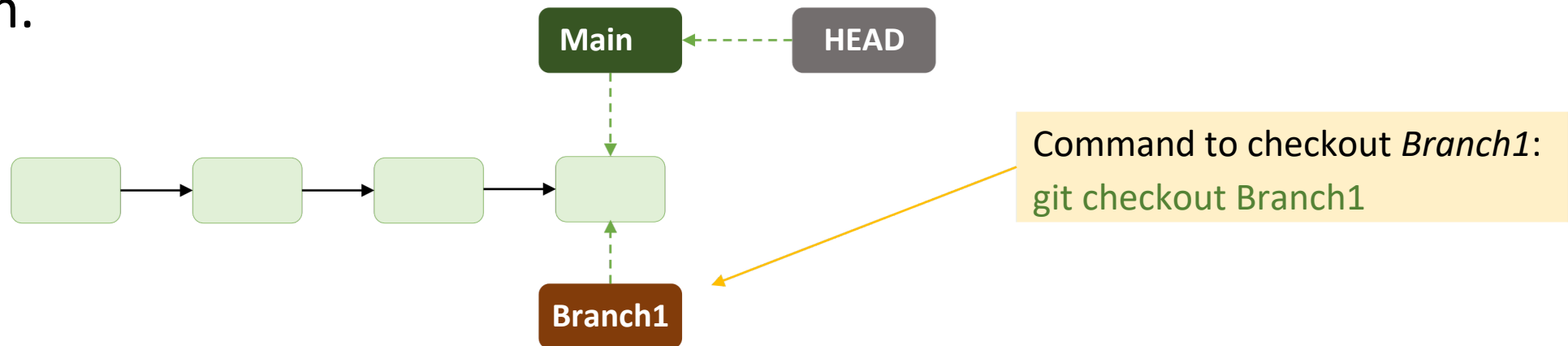
- Now how to switch to that new branch i.e. *Branch1*?
- *git checkout* command can be used to do so.
 - *git checkout Branch1*



Branching –working on new branch

Branch

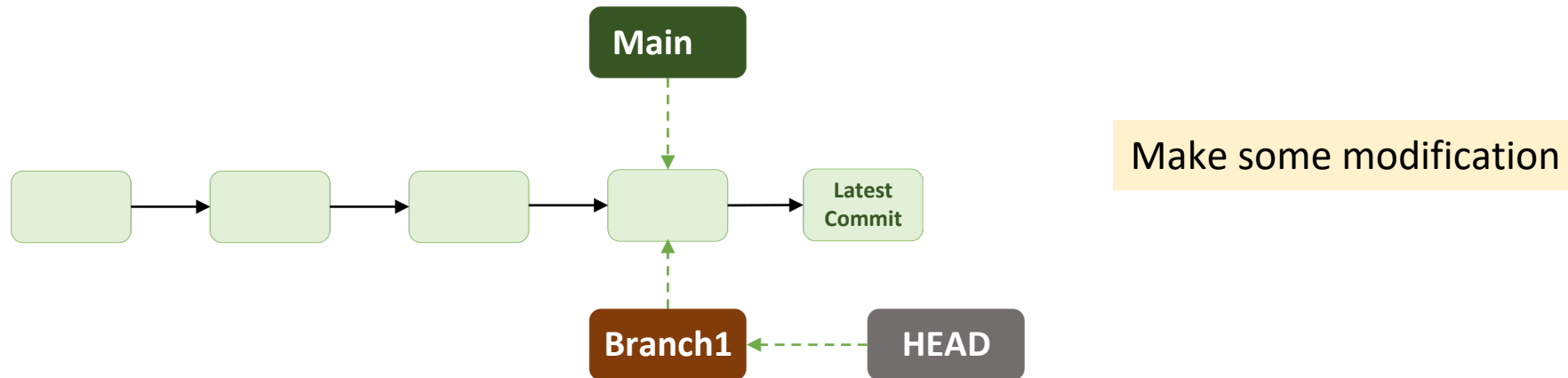
- *HEAD* is a *special pointer* to know what branch you are currently working on?
- When you start a new repo, *HEAD* pointer always points to *master* branch
- Upon checkout to different branch, *HEAD* pointer points to the new branch.



Branching –working on new branch

Branch

What happen when we make some changes to *Branch1*?



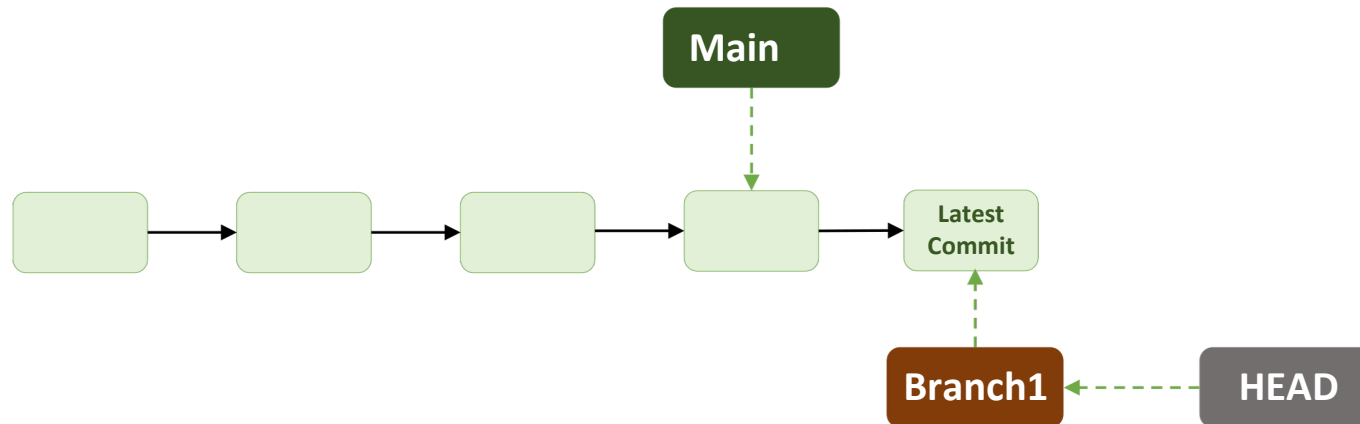
Branching –working on new branch

Branch

What happen when we make some changes to *Branch1*?

DIY: Try running (with multiple branches and multiple commits)

git log --all --graph



Pulling vs Cloning

Pulling vs cloning Repository

- **git clone:**

- To get a local copy of an existing repository to work on
- Clones a repository into a newly created directory, creates remote-tracking branches for each branch in the cloned repository
- usually only used once for a given repository
- usually to get a clean copy

- **git pull** command:

- Incorporates changes from a remote repository into the current branch
- ***update*** the local copy with new commits from the remote repository.

Fork and Merge

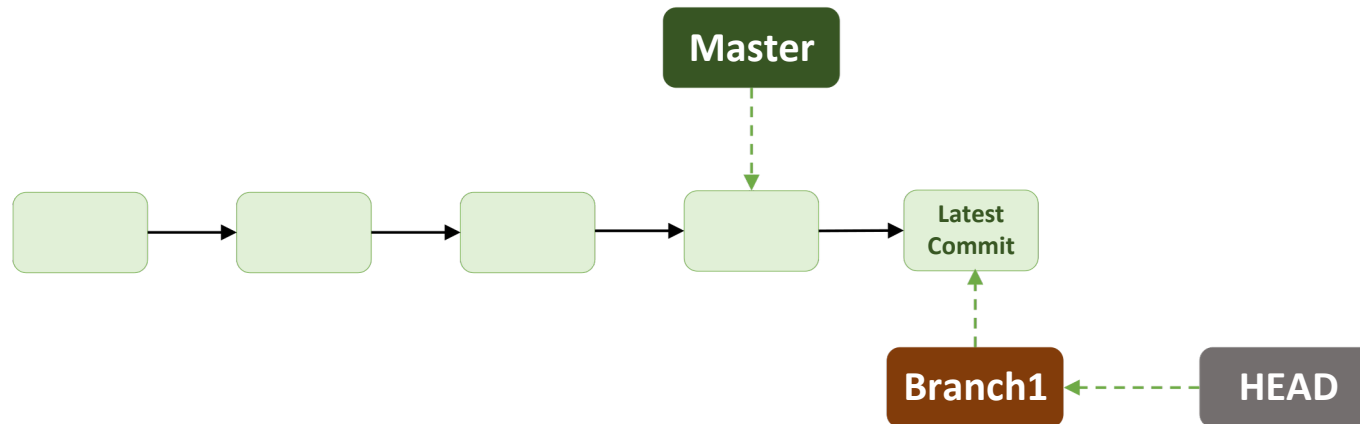
Forking

- Branching and forking provide two ways of diverging from the main code line.
- *Fork* is another way of saying clone or copy.
- Fork is a clone on the GitLab (or GitHub) side (it clones everything).
- If you fork a repository, you get that repository and all of its branches.
- A fork is independent from the original repository
- If the original repository is deleted, the fork remains.

Merging

In this case, lets merge *Branch1* and *Master* branch.

```
git checkout master
```



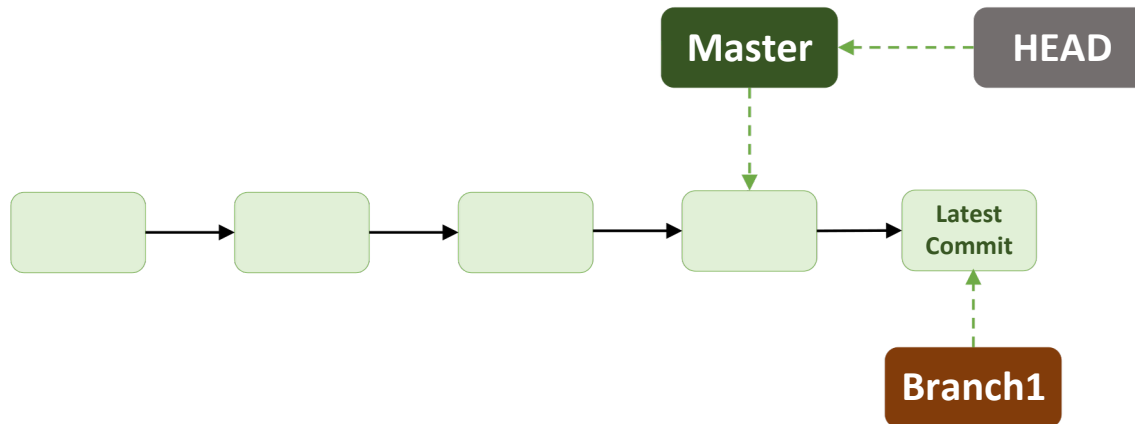
Merging

In this case, lets merge *Branch1* and *Master* branch.

```
git checkout master
```

```
git merge Branch1
```

Also known as fast-forward merging

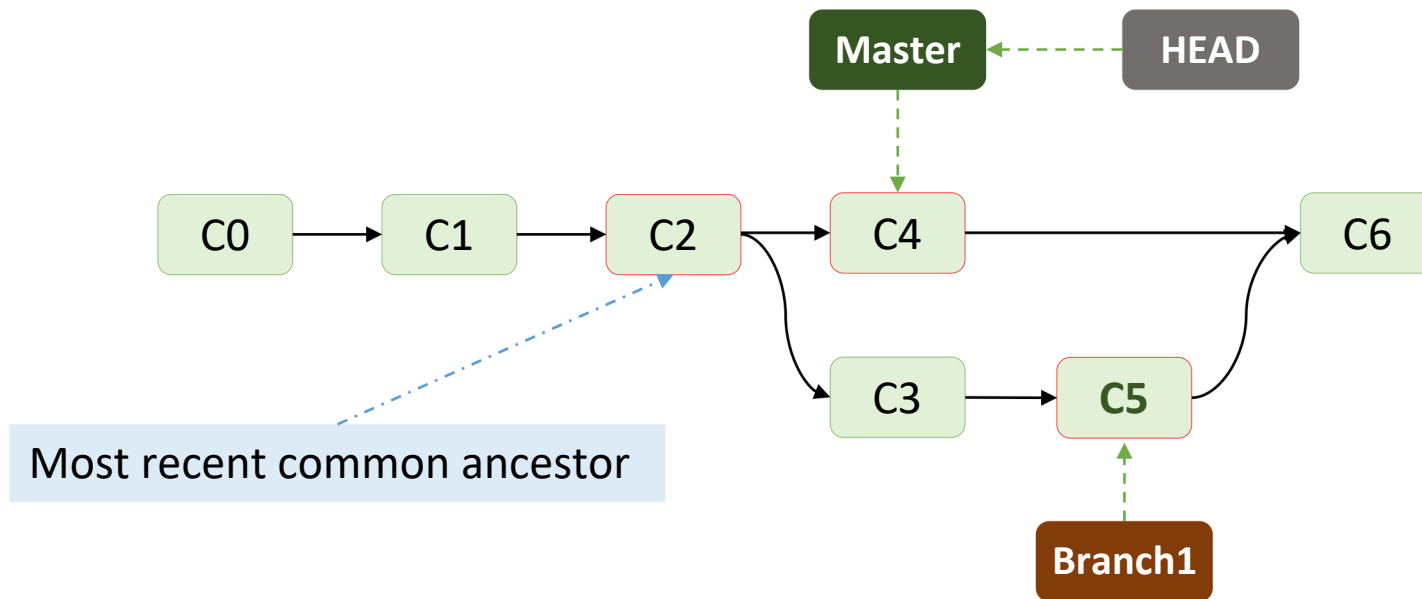


Merging

In this case, lets merge *Branch1* and *Master* branch.

```
git checkout master
```

```
git merge Branch1
```



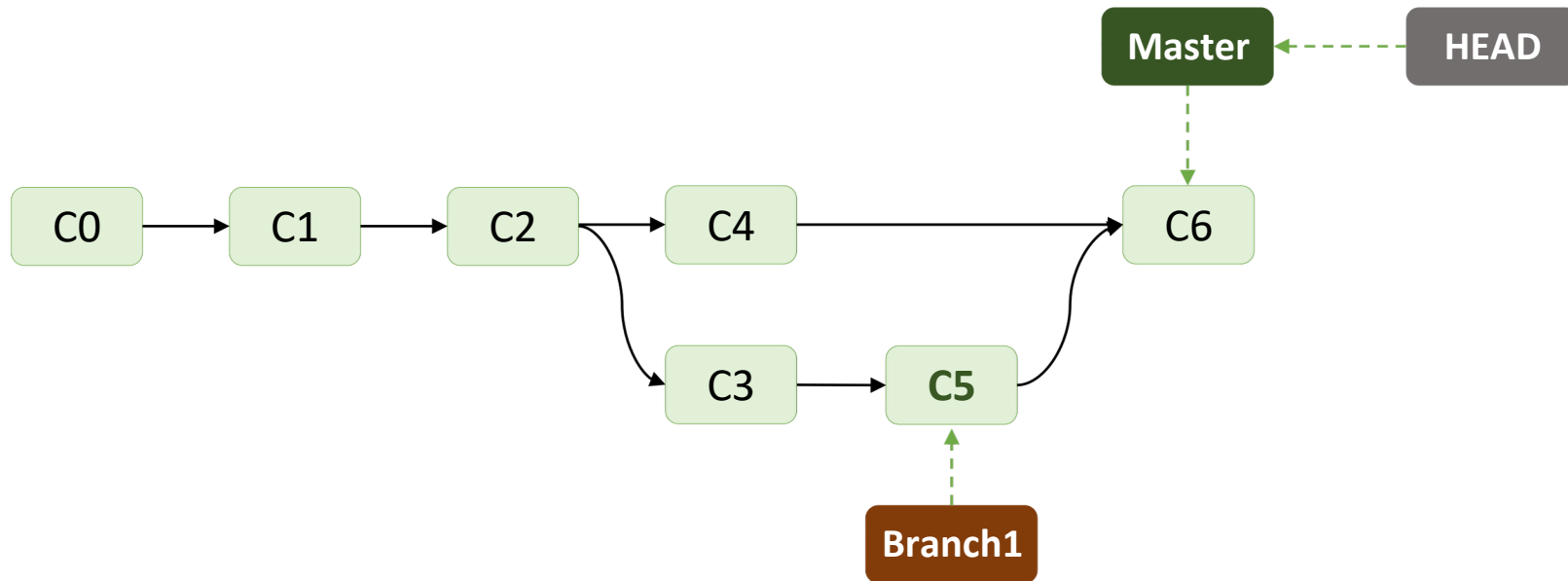
Merging

In this case, lets merge *Branch1* and *Master* branch.

```
git checkout master
```

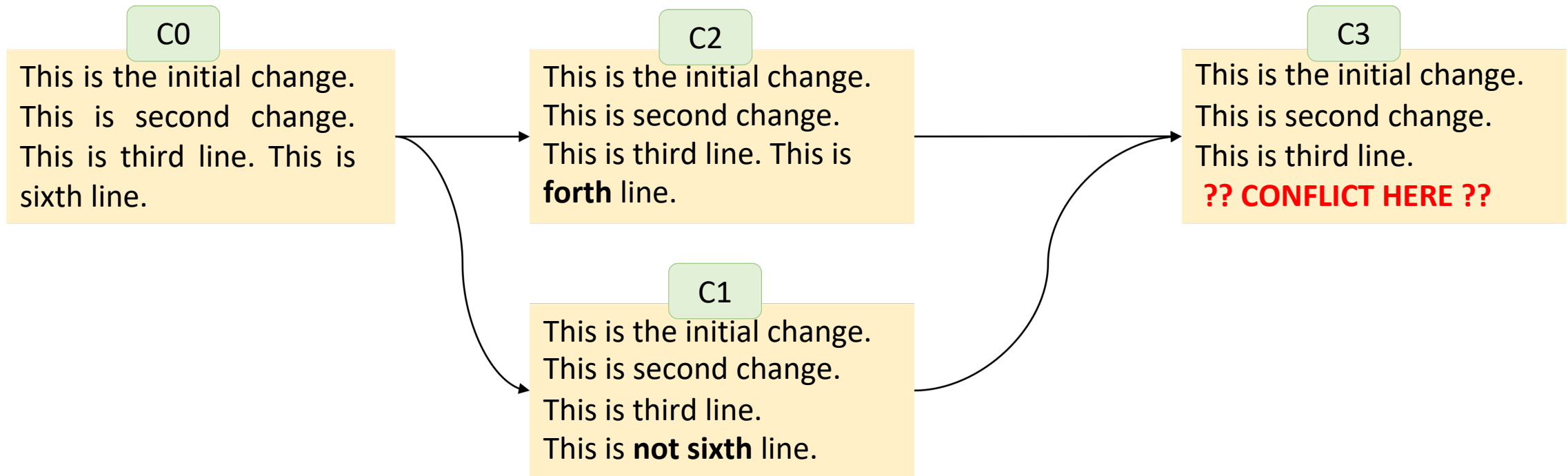
```
git merge Branch1
```

Also known as three-way merge



Merge conflict Example

- If you have changed the same part of the same file differently in the two branches

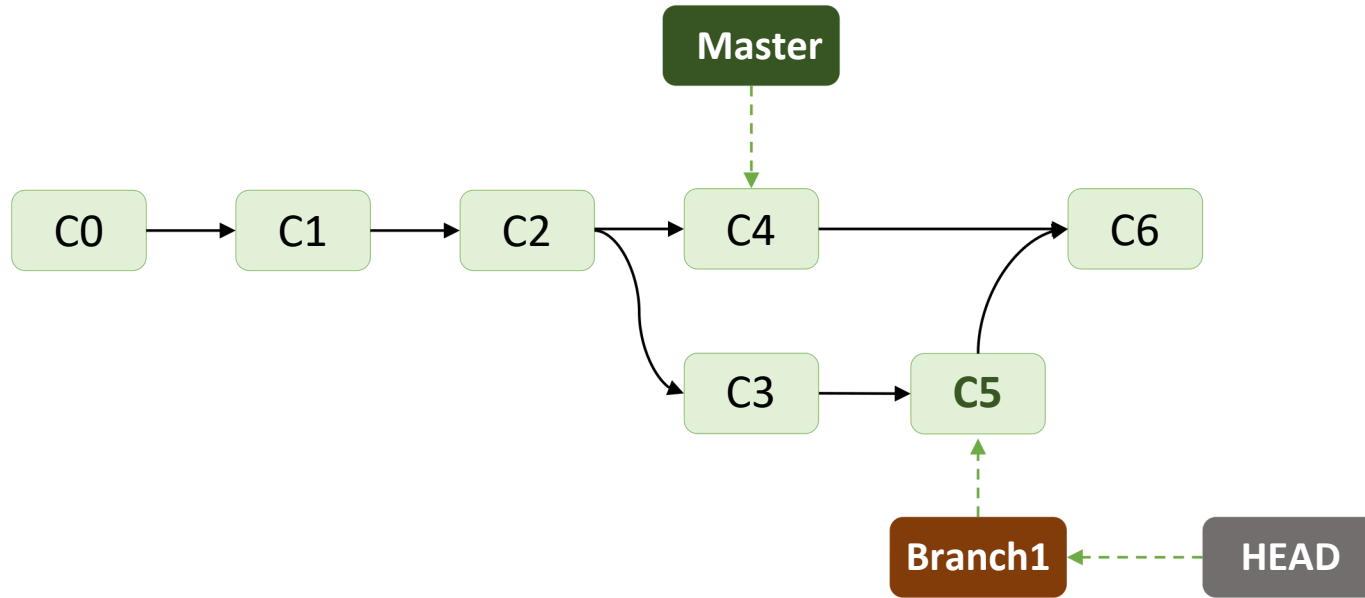


Merge + Rebase

- **Merge** when bringing changes from one branch to the master branch
- **Rebase** when updating your branch with master
- Common goal:
 - to integrate changes from one branch into another branch

Rebase

- **Useful:** when updating your branch with master



Take the patch of the change that was introduced in **C5** and reapply it on top of **C4**

commit the changes -> **C6**

References

- <https://bitbucket.org/product/version-control-software>
- <https://git-scm.com/> <https://gitlab-org.gitlab.io/>
<https://www.atlassian.com/git/tutorials>
- <https://www.htown-tech.com/blogs/centralized-vs-distributed-version-control-system>
- <https://gist.github.com/lifuzu/9490352>

Presentation Topics

1. Git Workflow Models
2. Comparing Git Hosting Platforms